Phase 1:

AI used: Gemini

Heres my initial prompt:

> I am working on a project involving java code to make a game. it is the "21 game". it should be two players (user vs computer) where the game starts with 21 and each player selects to remove 1, 2, or 3 from the total. the goal is to get to 0. whichever player gets to 0 first wins.

```
import java.util.Scanner;
import java.util.Random;

public class GameOf21 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Random random = new Random();

        int total = 21;
        boolean isPlayerTurn = true; // Start with the player

        System.out.println("Welcome to the 21 Game! Reach 0 to win.");

        while (total > 0) {
            System.out.println("\nCurrent total: " + total);
            int move = 0;

            if (isPlayerTurn) {
                // User's Turn
                while (true) {
                    System.out.print("Your turn (1-3): ");
                    move = scanner.nextInt();
                    if (move >= 1 && move <= 3 && move <= total) break;
                    System.out.println("Invalid move. Choose 1, 2, or 3.");
                }
            } else {
                // Computer's Turn (Randomized for now)
                move = random.nextInt(3) + 1;
                if (move > total) move = total; // Don't overshoot 0
                System.out.println("Computer chooses: " + move);
            }
```

```
            total -= move;

            if (total == 0) {
                String winner = isPlayerTurn ? "You" : "The Computer";
                System.out.println("\n" + winner + " hit 0 and wins!");
            }

            isPlayerTurn = !isPlayerTurn; // Switch turns
        }
        scanner.close();
    }
}
```

Phase 2:

Req. audit:

- Human vs computer opponent
  - Good - the COM player could be a little smarter of an opponent but it varies its approach enough so its not super predictable.
- Computer makes moves automatically (not second human input)
  - Good - read notes above
- Input validation (invalid moves don't crash the program)
  - Good - rejects inputs that are not 1, 2, or 3
- Replay without restarting
  - Nope - Game does not restart or prompt a request to restart.

Recommended minimum changes:

- Have gemini add a "New Game?" prompt at the end of the code.

Phase 3:

1. GameOf21 -Problem: (God Class)
   - One class handles most of the game panning from user input, COM logic, game state, and even victory conditions. The code really should have these separated into separate classes to prevent overstepping and other problems
2. Main - Problem: (Mixing I/O with the game logic)
   - The math logic is located between system.out.println and scanner.nextInt(). This kinda makes it difficult to test unless you manually do it yourself. It should be moved into its own method.
3. 21,1, 2, 3 - Problem: (Magic Numbers)

- If the game was to have its parameters changed this could be a problem and would have to be manually changed for each variable. Could make the qualifiers their own class/object.

4. “if (move >= 1 && move <= 3 && move <= total) break;” - Problem: (Duplicate Validation Logic)
    - Only the player 1’s moves are validated. It doesnt really check if the COM or potentially another player is using the correct numbers. This can be resolved by making simple check.
5. isPlayerTurn - Problem: (Tight Coupling)
    - No room for option for a human player. The instructions only want COM opponan so not really a problem but if the game wants to be modified itd need to be rewritten. Having a toggle for COM or hum P2 would work (probably wont do that though)
6. No play again option
    - Game ends after playing. It need to have a “play again?” prompt at the end

Phase 4:

Refractor 1:

Prompt:

> the code is great but could use some minor tweaks. First lets add a "play again?" option at the end. dont alter anything else unless its to allow the player to request to play again after a winner is decided.

Results:

```
*** You hit 0 and wins! ***

Would you like to play again? (yes/no): yes

--- New Game: Welcome to 21! ---

Current total: 21
Your turn (1-3):
```

Refractor 2:

Prompt:

excellent job. now lets make the system check the computer input jut to make sure its not using anything other than a valid input. add some type of catch exception for the computer opponent.

Results:

```
} catch (IllegalMoveException e) {
    // This "catches" the computer if it tries to cheat or errors out
    System.err.println("Game Error: " + e.getMessage());
    move = Math.min(total, 1); // Fallback safety move
    System.out.println("System corrected move to: " + move);
    total -= move;
    isPlayerTurn = !isPlayerTurn;
}
```

Makes sure the computer can't cheat

Refraction 3:

PRompt:

great work. lets work on the God class problem. GameOf21 has too many functions. it should have separate functions or classes for user input, computer player logic, game state, and victory conditions.

Result:

```
interface Player {  7 usages  2 implementations
    int getMove(int currentTotal);  1 usage  2 impl
    String getName();  1 usage  2 implementations
}

// 3. Human Player
class HumanPlayer implements Player {  1 usage
```

```java
class ComputerPlayer implements Player {  1 usage
    private final Random random = new Random();  1 usage

    @Override  1 usage
    public String getName() { return "Computer"; }

    @Override  1 usage
    public int getMove(int total) {
        int maxMove = Math.min(total, 3);
        int move = random.nextInt(maxMove) + 1;
        System.out.println("Computer chooses: " + move);
        return move;
    }
}
```

```java
class GameSession {  2 usages
    private int total;  5 usages
    private static final int STARTING_TOTAL = 21;  1 usage

    public GameSession() { this.total = STARTING_TOTAL; }  1 usage
    public boolean isGameOver() { return total <= 0; }  2 usages
    public int getTotal() { return total; }  2 usages

    public void subtract(int amount) throws IllegalMoveException {  1 usage
        if (amount < 1 || amount > 3 || amount > total) {
            throw new IllegalMoveException("Illegal move: " + amount);
        }
        total -= amount;
    }
```

Separates everything into separate classes for easy modification and less spaghetti code.

Refracture 4: (FINAL)

great work. last thing. there shouldnt be any game logic in main. and the computer player should be a bit smarter (i.e. it should be able to recognize if the remaining amount is one of the valid moves it can make and ensure it tries to win in that state.

Results:

```java
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        GameManager manager = new GameManager(scanner);
        boolean playAgain;

        do {
            manager.startNewGame();
            System.out.print("\nPlay again? (y/n): ");
            playAgain = scanner.next().equalsIgnoreCase( anotherString: "y");
        } while (playAgain);

        System.out.println("Thanks for playing!");
        scanner.close();
    }
```

```
Current Total: 5
Your turn (1-3): 3

Current Total: 2
Computer chooses: 2

*** Computer wins! ***

Would you like to play again? (y/n):
```

The computer now is smart enough to actually try to win near the end. Also Main is clean of any game logic.

Finalized Code:

```
import java.util.Scanner;
import java.util.Random;
import java.util.InputMismatchException;

// 1. Custom Exception
class IllegalMoveException extends Exception {
    public IllegalMoveException(String message) { super(message); }
}

// 2. Player Interface
interface Player {
    int getMove(int currentTotal);
    String getName();
}

// 3. Human Player
class HumanPlayer implements Player {
    private final Scanner scanner;
    public HumanPlayer(Scanner scanner) { this.scanner = scanner; }
    @Override
    public String getName() { return "You"; }

    @Override
    public int getMove(int total) {
        while (true) {
            try {
                System.out.print("Your turn (1-3): ");
                int move = scanner.nextInt();
                if (move >= 1 && move <= 3 && move <= total) return move;
                System.out.println("Invalid! Choose 1-3 (Remaining: " + total + ")");
            } catch (InputMismatchException e) {
                System.out.println("Error: Enter a number.");
                scanner.next();
            }
        }
    }
}

// 4. Smart Computer Player
class ComputerPlayer implements Player {
    private final Random random = new Random();
    @Override
    public String getName() { return "Computer"; }
```

```
        @Override
        public int getMove(int total) {
            int move;
            // Smart Check: Can I win right now?
            if (total <= 3) {
                move = total;
                System.out.println("Computer sees victory!");
            } else {
                // Otherwise, pick randomly (or use math strategy)
                move = random.nextInt(3) + 1;
            }
            System.out.println("Computer chooses: " + move);
            return move;
        }
    }

    // 5. Game State
    class GameSession {
        private int total = 21;
        public boolean isGameOver() { return total <= 0; }
        public int getTotal() { return total; }
        public void subtract(int amount) throws IllegalMoveException {
            if (amount < 1 || amount > 3 || amount > total) throw new
IllegalMoveException("Illegal move");
            total -= amount;
        }
    }

    // 6. The Manager (Decouples Logic from Main)
    class GameManager {
        private final Scanner scanner;

        public GameManager(Scanner scanner) {
            this.scanner = scanner;
        }

        public void startNewGame() {
            GameSession session = new GameSession();
            Player human = new HumanPlayer(scanner);
            Player computer = new ComputerPlayer();
            Player currentPlayer = human;

            System.out.println("\n=== Game of 21: Reach 0 to Win ===");
```

```
        while (!session.isGameOver()) {
            System.out.println("\nCurrent Total: " + session.getTotal());
            try {
                int move = currentPlayer.getMove(session.getTotal());
                session.subtract(move);

                if (session.isGameOver()) {
                    System.out.println("\n*** " + currentPlayer.getName() + " wins! ***");
                } else {
                    currentPlayer = (currentPlayer == human) ? computer : human;
                }
            } catch (IllegalMoveException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}

// 7. Main (Only handles the Replay Loop and Initialization)
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        GameManager manager = new GameManager(scanner);
        boolean playAgain;

        do {
            manager.startNewGame();
            System.out.print("\nPlay again? (y/n): ");
            playAgain = scanner.next().equalsIgnoreCase("y");
        } while (playAgain);

        System.out.println("Thanks for playing!");
        scanner.close();
    }
}
```